Advanced Object Oriented Programming

Xingjie ZHANG

아주대학교
AJOU UNIVERSITY

아주대학교
AJOU UNIVERSITY

Array

Instructor: Xingjie ZHANG

Email : xzhang@ajou.ac.kr

Wechat : runningSnaile

CONTENTS

01 Array

02 Sorting

03 n-d Array

04 Lookup Array

아주대학교
AJOU UNIVERSITY

01

Array

Array

- A grammar for managing multiple data types as a single unit.
- Zero-based index.
- An array name is a reference to an array object.
- In Java, arrays are implemented as objects.

Array

- _arrayNeeds

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int a, b, c;
        a = s.nextInt();
        b = s.nextInt();
        c = s.nextInt();
        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
    }
}
```

Array

byte[] array

char array[]

int[] array

Foreach loop

- _forEach

```
public class Main {
    public static void main(String[] args) {
        int[] list = {10, 20, 30, 40, 50};
        for(int i = 0; i < list.length; ++i) {
            System.out.println("list[" + i + "] " + list[i]);
        }

        for (int i : list) {
            System.out.println(i);
        }
    }
}
```

Array

- _arraySample

```
public class Main {
    public static void main(String[] args) {
        //int aData[] = {10, 20, 30, 40, 50};
        int[] array = {10, 20, 30, 40, 50};
        int idx = 0;
        System.out.println("array length: " + array.length);
        for(int data : array) {
            System.out.println("array[" + idx + "] " + data);
            idx++;
        }
    }
}
```

Array

- _arrayAccess

```
public class Main {
    public static void main(String[] args) {
        int[] array = new int[] {10, 20, 30, 40, 50};

        array[0] = 100;
        array[3] = 200;
        for (int i = 0; i < array.length; i++) {
            System.out.printf("array[%d]: %d\n", i, array[i]);
        }
    }
}
```

Character Array

- The actual string is char[].
- A **string constant** can be interpreted as an **unnamed char[]** that is not writable (String Class).

Character Array

- _charArray

```
public class Main {
    public static void main(String[] args) {
        char[] hello = new char[5];
        String helloString = "Hello";
        for (int i = 0; i < 5; i++) {
            hello[i] = helloString.charAt(i);
            System.out.printf("hello[%d]: %c\n", i, hello[i]);
        }
    }
}
```

Array

- Finding the maximum value - Array

50 40 10 30 20

MAX: 50

02

Sorting

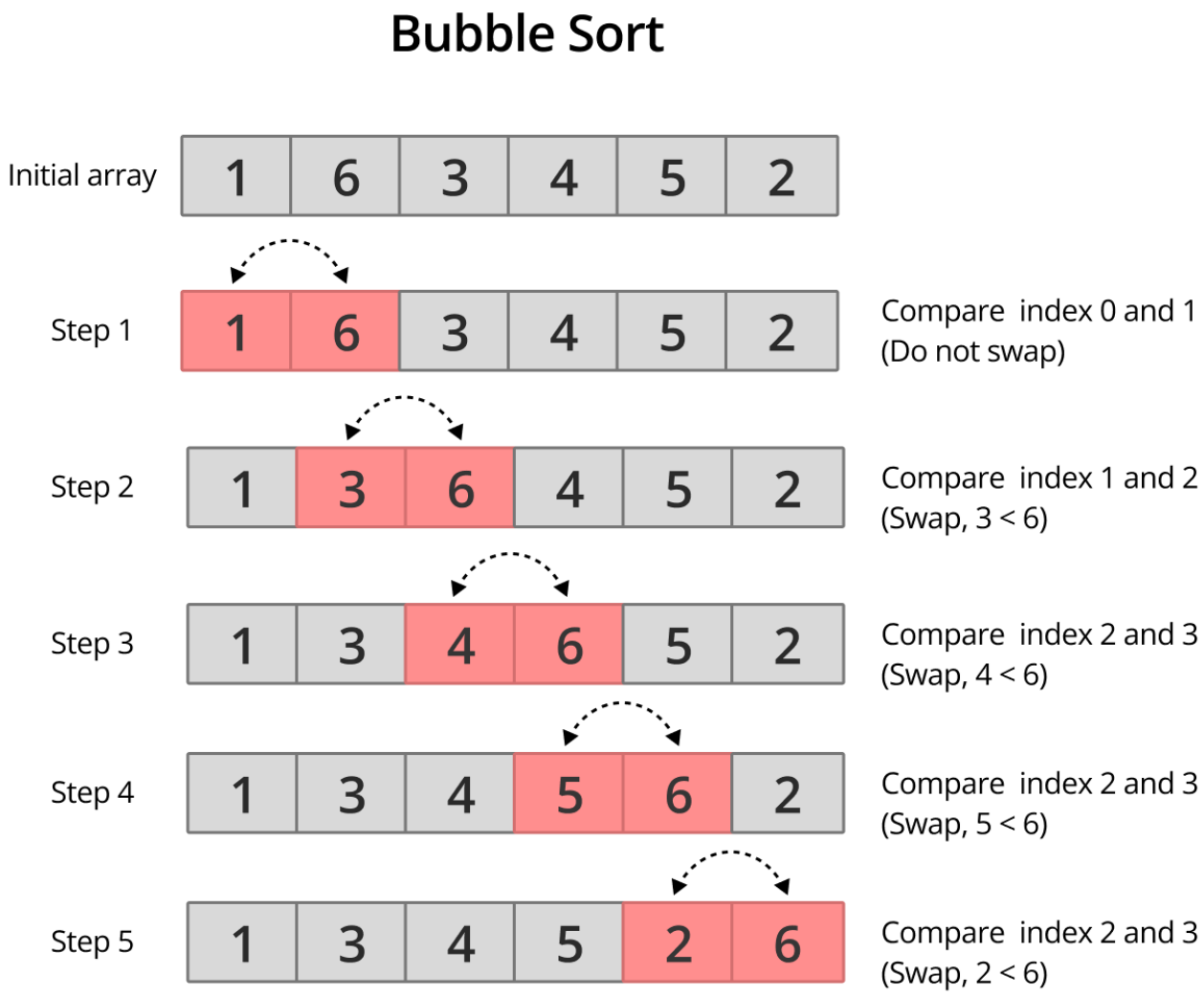
Sorting Algorithm

- What is a Sorting Algorithm?
  - A **sorting algorithm** is an algorithm that rearranges data in a specific order (ascending or descending).
- Purpose
  - Organizes data systematically to facilitate searching, analysis, and processing
  - Improves efficient data management and accessibility
- Sorting Criteria
  - **Ascending Order**: From smallest to largest value (e.g., 1, 2, 3, 4, 5)
  - **Descending Order**: From largest to smallest value (e.g., 5, 4, 3, 2, 1)

Sorting Algorithm

- Major Types of Sorting Algorithms
  - Simple Sorting Algorithms
    - **Bubble Sort**: Compares and swaps adjacent elements
    - **Selection Sort**: Finds the minimum value and moves it forward
    - **Insertion Sort**: Inserts elements into their correct position
  - Efficient Sorting Algorithms
    - **Quick Sort**: Divide and conquer using a pivot
    - **Merge Sort**: Divides the array and merges while sorting
    - **Heap Sort**: Sorts using a heap data structure
  - Specialized Sorting Algorithms
    - **Counting Sort**: Sorts by counting the frequency of integer values
    - **Radix Sort**: Sorts by individual digits
    - **Bucket Sort**: Distributes data into buckets and sorts

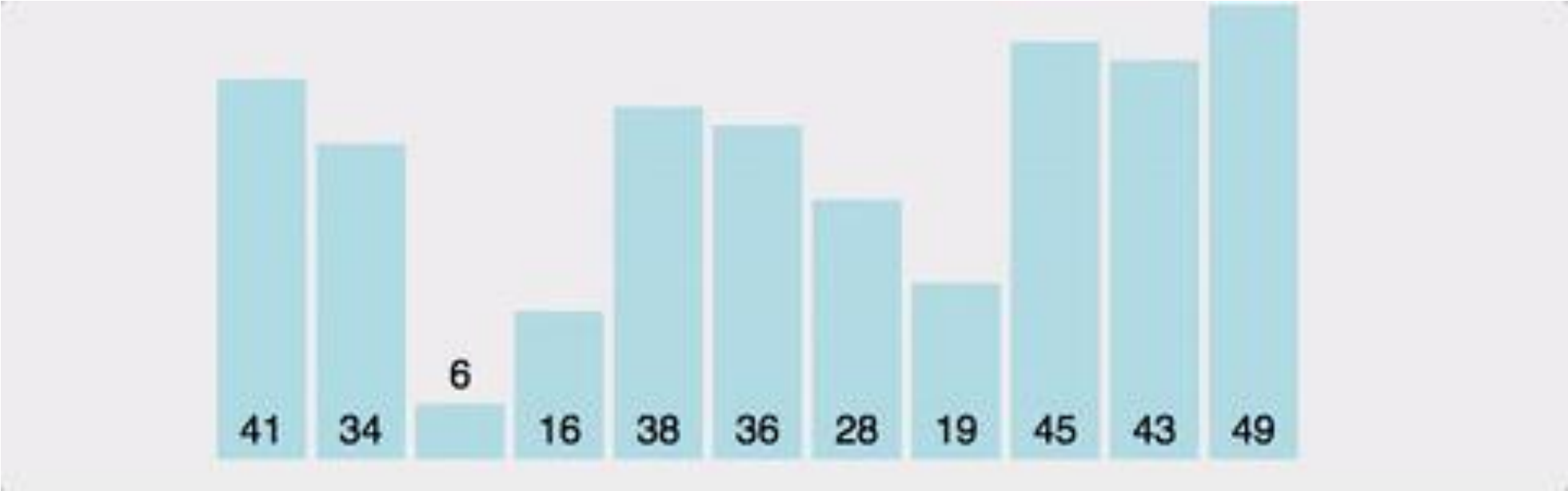
Bubble Sort



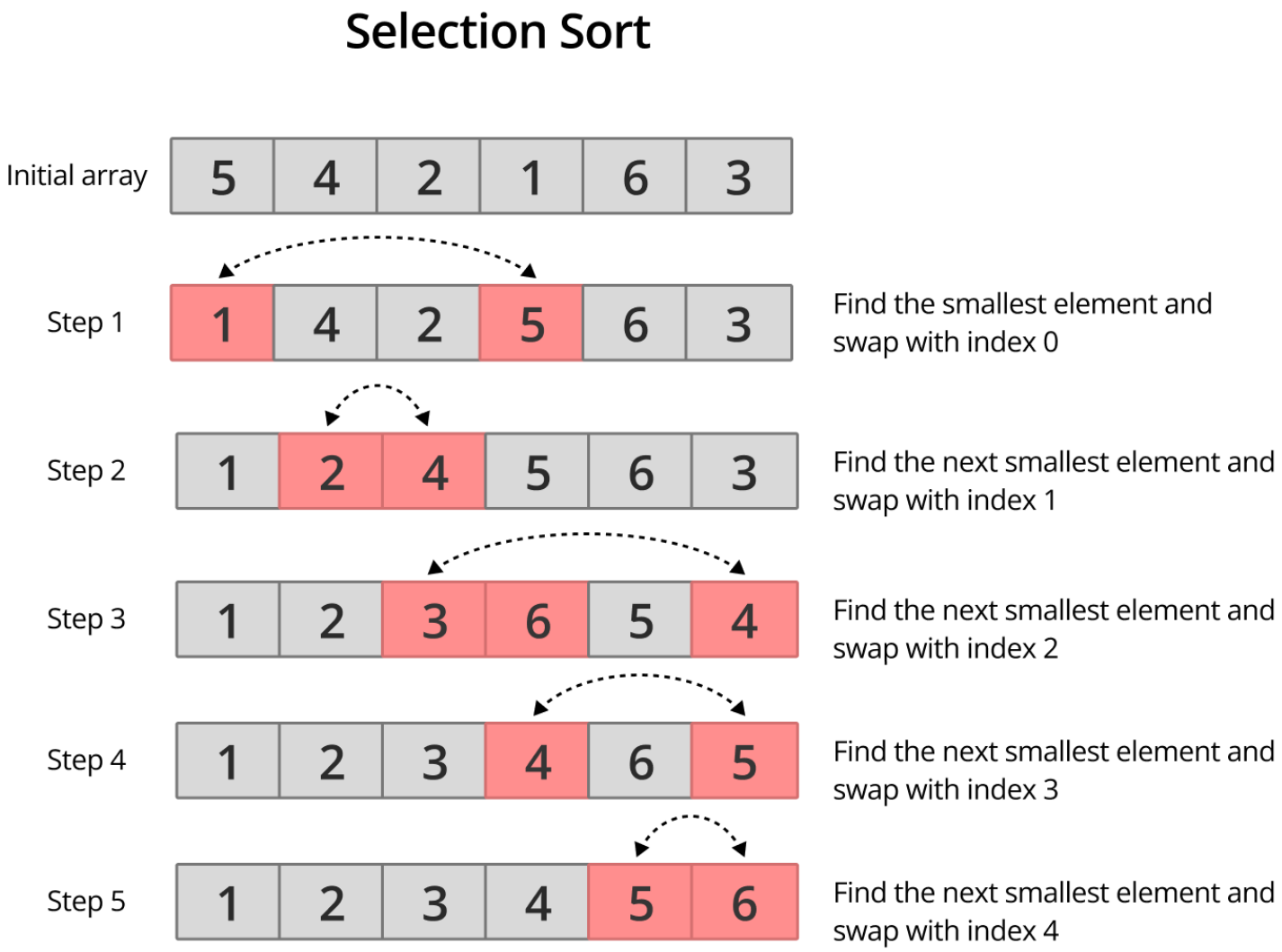
Repeat the steps starting from the beginning until no items are swapped

Bubble Sort

```
for pass = 1 to n-1          // Perform (n-1) passes
   for i = 1 to n-pass       // Compare adjacent elements from A[0] to A[n-pass]
      if A[i-1] > A[i]      // If the element on the left is greater than the one on the right
         swap A[i-1] and A[i] // Swap their positions
return array A
```

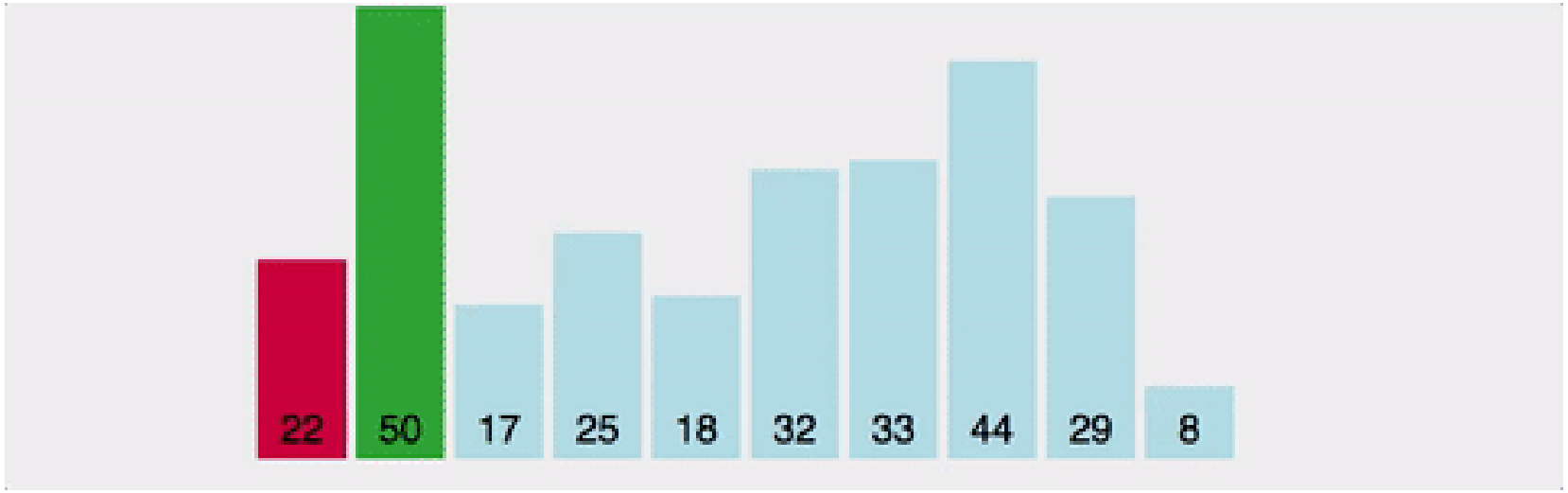


Selection Sort

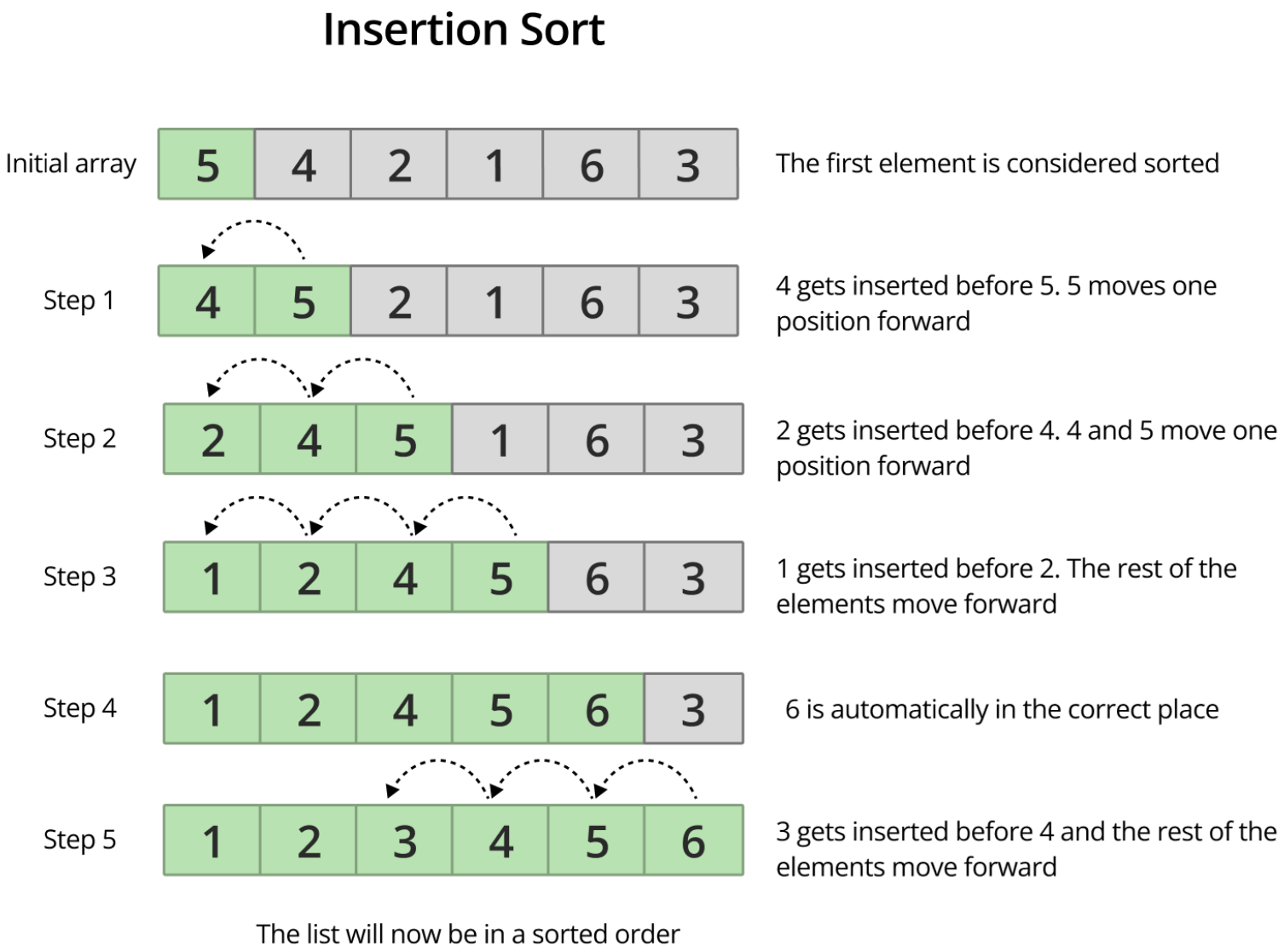


Selection Sort

```
for i = 0 to n-2                // Iterate from the first to the second-to-last element
   min_index = i                // Assume the current position is the minimum
   for j = i+1 to n-1           // Search for the minimum in the unsorted part
      if A[j] < A[min_index]    // If a smaller element is found
         min_index = j          // Update the minimum index
   if min_index != i            // If the minimum is not at the current position
      swap A[i] and A[min_index]  // Swap the found minimum with the current element
return array A
```

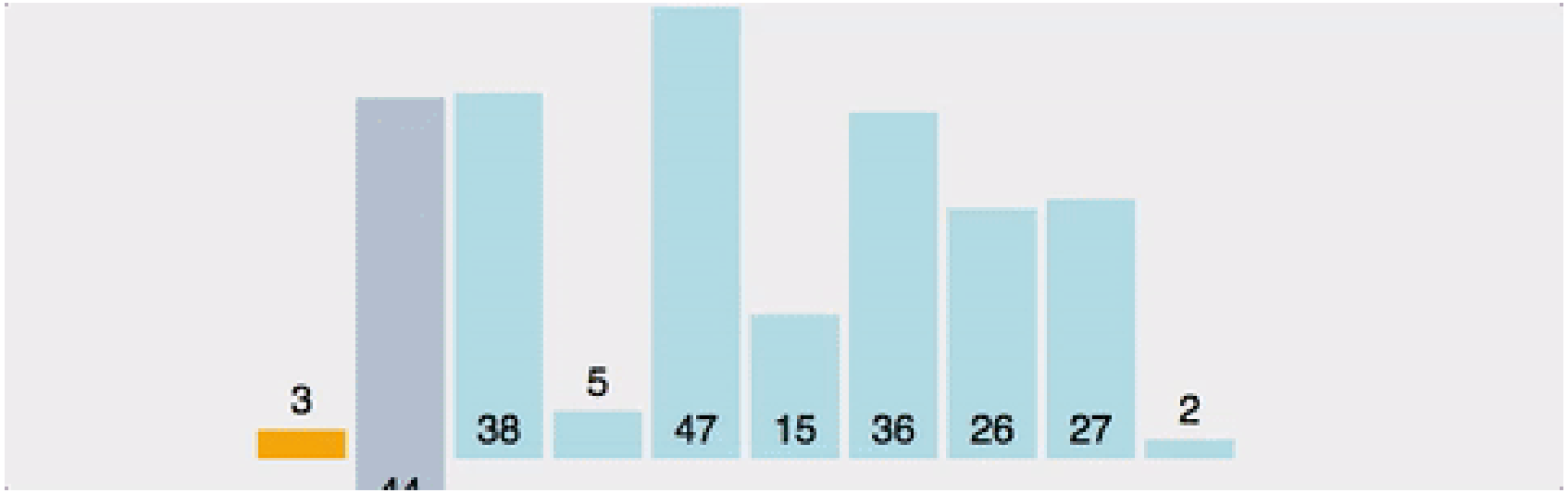


Insertion Sort



Insertion Sort

```
for i = 1 to n-1                // Start from the second element
   key = A[i]                   // Store the current element as key
   j = i - 1                    // Set j to the index before i
   while j >= 0 and A[j] > key  // Move elements greater than key to one position ahead
      A[j + 1] = A[j]
      j = j - 1
   A[j + 1] = key               // Insert the key at the correct position
return array A
```



03

n-d Array

n-d Array

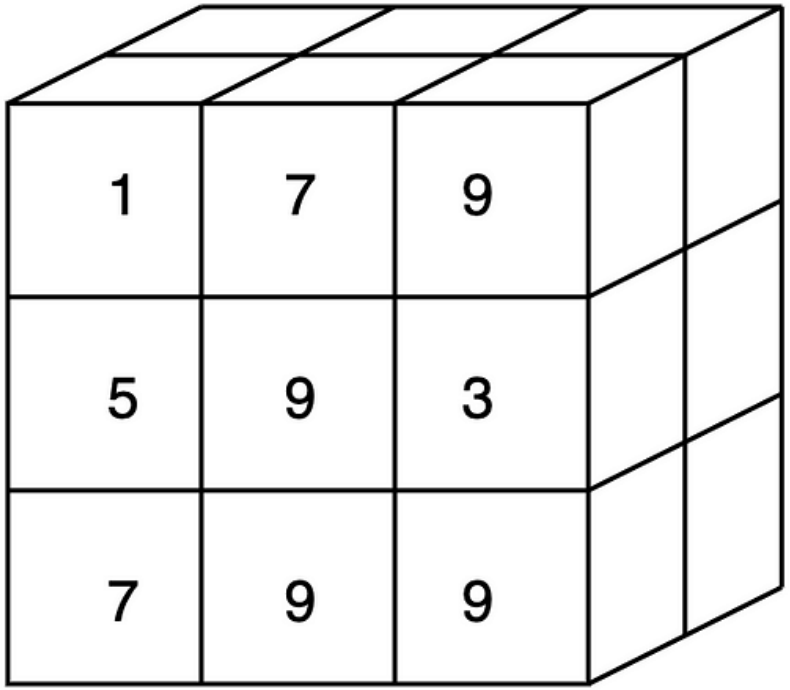
1D Array

3	2

2D Array

1	0	1
3	4	1

3D Array



n-d Array

- _multiArray

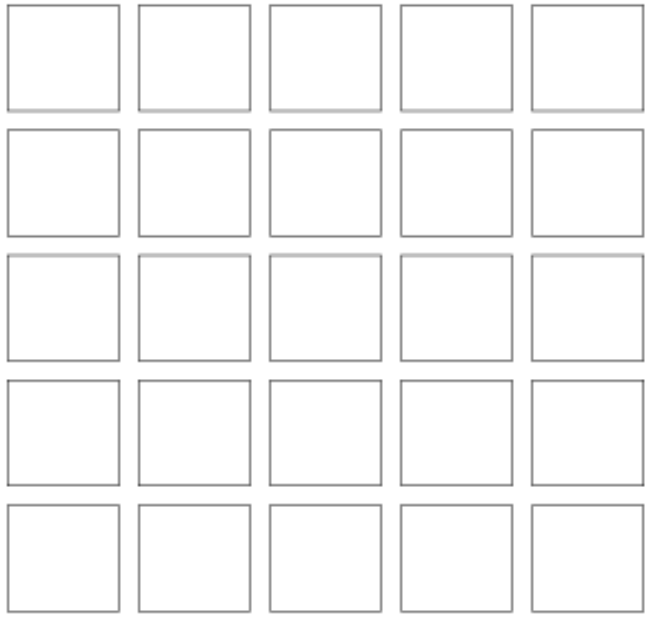
```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[][] array = {
            {10, 20, 30, 40},
            {50, 60, 70, 80},
            {90, 100, 110, 120}
        };

        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array[i].length; j++) {
                System.out.printf("%d\t", array[i][j]);
            }
            System.out.print('\n');
        }
    }
}
```

n-d Array

- Sequential filling of a 2-d array



1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

n-d Array

- _multiArrayCounter

```
public static void main(String[] args) {
    int[][] array = new int[5][5];

    int cnt = 0;
    for (int i = 0; i < array.length; i++) {
        for (int j = 0; j < array[i].length; j++)
            array[i][j] = ++cnt;
    }

    for (int i = 0; i < array.length; i++) {
        for (int j = 0; j < array[i].length; j++) {
            System.out.printf("%d\t", array[i][j]);
        }
        System.out.print('\n');
    }
}
```

04

Lookup Array

Lookup Array

- An array used to search or reference data.
- A structure that calculates an index value based on criteria and utilizes the identified element.
- A single element in the array is used as a single case to be processed.
- **A method that dramatically improves the shortcomings of the switch-case structure.**

n-d Array

- _busFareWithLookupArray

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        double[] rateArray = {
                0.0, 0.1, 0.25,
                0.5, 0.5,
                0.6, 0.65,
                0.8, 0.82, 0.97
        };

        final int fee = 1000;
        System.out.println("bus fare");
        for (int i = 1; i <= 10; i++) {
            System.out.printf("age: %d bus fare: \t%d\n",
                    i, (int)(fee * rateArray[i - 1]));
        }

        System.out.print("plz enter your age: ");
        int age = s.nextInt();
        if(age < 1)
            age = 1;
        else if(age > 10)
            age = 10;

        System.out.printf("bus fare: %d\n",
                (int)(fee * rateArray[age - 1]));
    }
}
```

Shallo copy and Deep copy

- In Java, arrays are implemented as **objects**.
- Identifiers for array objects are ultimately references.
- Multiple references can exist to a single object.
- Beware of deep copy and shallow copy issues.

Shallo copy

- _arrayShallowCopy

```
public class Main {
    public static void main(String[] args) {
        int[] aSrc = {10, 20 , 30, 40, 50};
        int[] aDst = aSrc;

        aDst[2] = 100;
        for(int data : aSrc)
            System.out.print(data + "\t");
    }
}
```

Deep copy

- _arrayDeepCopy

```
public class Main {
    public static void main(String[] args) {
        int[] aSrc = {10, 20 , 30, 40, 50};

        int[] aDst01 = new int[aSrc.length];
        for (int i = 0; i < aSrc.length; i++) {
            aDst01[i] = aSrc[i];
        }

        aDst01[2] = 100;

        int[] aDst02 = aSrc.clone();
        aDst02[3] = 300;
        System.out.print("\n  aSrc[]: ");
        for(int data : aSrc)
            System.out.print(data + "\t");

        System.out.print("\naDst02[]: ");
        for(int data : aDst02)
            System.out.print(data + "\t");
    }
}
```

Deep copy

- **System.arraycopy()** function

```
public class Main {
    public static void main(String[] args) {
        int[] aSrc = {10, 20 , 30, 40, 50};
        int[] aDst = new int[aSrc.length];

        System.arraycopy(aSrc, 0, aDst, 0, aDst.length);

        aSrc[2] = 100;
        System.out.print("aSrc[]: ");
        for(int data : aSrc)
            System.out.print(data + "\t");

        System.out.print("\naDst[]: ");
        for(int data : aDst)
            System.out.print(data + "\t");
    }
}
```

Quiz

- 1. What data structure is used to efficiently manage multiple data items of the same type?
  - A. Variable
  - B. Object
  - C. Method
  - D. Array

Quiz

- 2. From which number does the index used to access elements of a Java array start?
  - A. 1
  - B. Array length
  - C. 0
  - D. Any number

Quiz

- 3. In which memory area is a Java array object allocated when it is created?
  - A. Stack area
  - B. Register
  - C. Heap area
  - D. Data area

Quiz

- 4. What does the .length member of a Java array object represent?
  - A. The value of the first element of the array
  - B. The last index of the array
  - C. The total sum of the array elements
  - D. The number of elements stored in the array

Quiz

- 5. What does a String in Java fundamentally group together?
  - A. A collection of numbers
  - B. An array of characters
  - C. A list of objects
  - D. A sequence of boolean values

Quiz

- 6. What is one major difference between bubble sort and selection sort?
  - A. Number of comparisons
  - B. Amount of memory required
  - C. Number of swaps per pass
  - D. Types of data that can be sorted

Quiz

- 7. Conceptually, how is a two-dimensional array structured in Java?
  - A. An array with arrays as its elements
  - B. A continuous structure of a single dimension
  - C. A structure that stores only objects
  - D. A structure that stores only sorted data

Quiz

- 8. In a two-dimensional array like int[][] arr;, what does arr.length represent?
  - A. Total number of elements
  - B. Number of columns in a specific row
  - C. Number of rows in the array
  - D. Number of dimensions of the array

Quiz

- 9. What is the main advantage of using a lookup array?
  - A. Reduces code writing time
  - B. Replaces complex conditional statements (if/switch) and improves performance
  - C. Reduces memory usage
  - D. Provides data encryption functionality

Quiz

- 10. When assigning an array variable in Java like arr2 = arr1;, what type of array copy occurs?
  - A. Deep copy
  - B. Reference copy
  - C. Element-wise copy
  - D. Structural copy

Practice

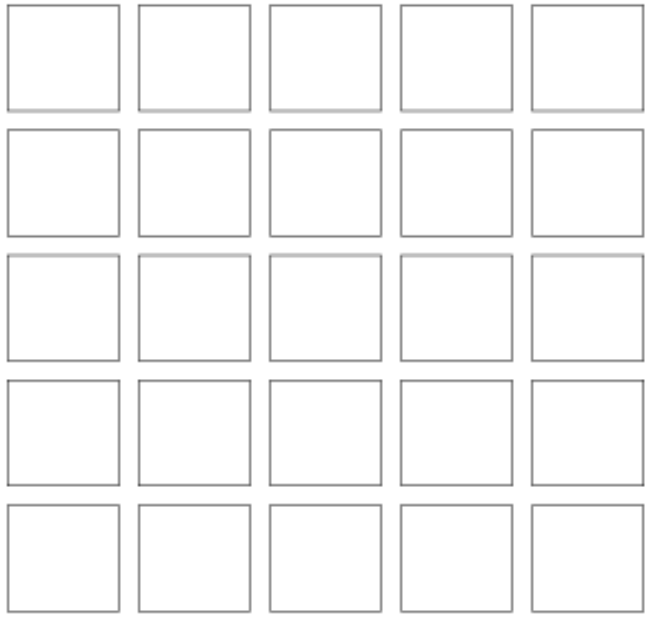
- 求出行和列的和

```
int [][]array = {
        {10, 20, 30, 0},
        {40, 50, 60, 0},
        { 0,  0,  0,  0}
};
```

```
10 20 30 60
40 50 60 150
50 70 90 210
```

Practice

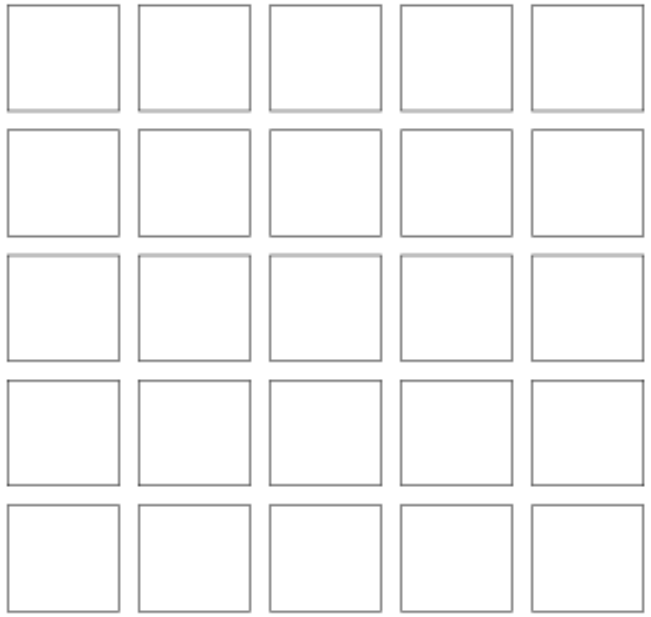
- 利用循环对一个空数组进行交叉填充



1	2	3	4	5
10	9	8	7	6
11	12	13	14	15
20	19	18	17	16
21	22	23	24	25

Practice

- 利用循环对一个空数组进行旋转填充



1	2	3	4	5
16	17	18	19	6
15	24	25	20	7
14	23	22	21	8
13	12	11	10	9

Practice

- 写出以下三种排序算法
  - 冒泡排序
  - 选择排序
  - 插入排序

- 使用验证数据集进行验证

Q & A

THANK YOU